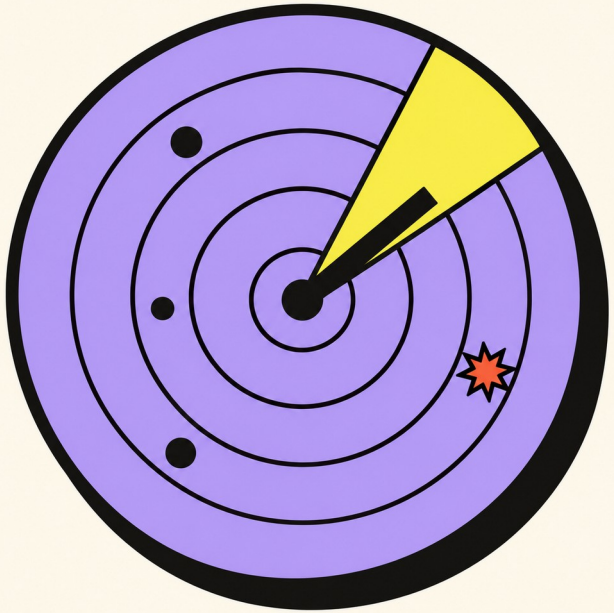
ИНСТРУКЦИЯ ДЛЯ НОВОЙ УСТАНОВКИ

РАЗВЁРТЫВАНИЕ И ПОДКЛЮЧЕНИЕ СЕРВИСОВ

Технологический радар
Руководство для администратора



Настройки, ключи API и модели • 27 сентября 2026

СОДЕРЖАНИЕ

Выберите раздел. Примеры команд можно копировать из PDF.

1 Подготовка 1

2 Файл настроек 6

3 Подключение LLM 10

4 Ключи источников 18

5 Остальные модели 25

6 Запуск и проверка 34

7 Обслуживание 38

8 Справочник 46

Команды выполняйте в терминале.
Значения записывайте в infra/.env.
Начните с раздела «Подготовка».

КАКОЙ ВАРИАНТ ЗАПУСКА ВЫБРАТЬ

Вариант	Что вы получите
Только радар **make laptop**	Поиск технологий, отчёты и модель этапа 1. Работает на CPU. LLM подключается отдельно.
Полный комплекс **make bootstrap**	Радар и работа с документами, распознавание, темы, гипотезы и метрики. Без NVIDIA запускает модели на CPU.
Полный комплекс с NVIDIA **make up**	Тот же набор функций. Для запуска нужны драйвер NVIDIA и NVIDIA Container Toolkit.

В режиме laptop загрузка документов, темы и метрики недоступны. Для них нужен полный комплекс.

КАК ПОЛЬЗОВАТЬСЯ ЭТОЙ ИНСТРУКЦИЕЙ

1 **Команды**

Откройте терминал на машине, где будет работать комплекс. Выполняйте команды из папки проекта, где лежит Makefile.

---

2 **Настройки**

Строки вида ИМЯ=значение вставляйте в infra/.env. Текст в <угловых скобках> заменяйте своими данными вместе со скобками.

---

3 **Проверка результата**

После каждого подключения есть проверка. Переходите дальше, когда получили указанный результат. Секретные значения никому не отправляйте.

LLM пишет тексты. API-ключ даёт программе доступ к сервису. Контейнер запускает отдельную часть комплекса.

ПОДГОТОВКА МАШИНЫ

Терминал: папка проекта rag-test

ВЫПОЛНИТЬ В ТЕРМИНАЛЕ

```
docker --version
docker compose version
docker info
make --version
python3 --version
```

Что установить

Docker 24+ с Compose 2.24.4+, Git, make и Python 3. На сервере используйте Linux x86-64. Запустите Docker.

Память и диск

Для laptop выделите Docker не менее 12 ГБ памяти и около 40 ГБ диска. Для полного комплекса предусмотрите от 100 ГБ диска.

Модели и сеть

Первому запуску нужен интернет для образов и весов. Требования полной системы к памяти зависят от выбранных моделей.

Docker Desktop: Settings → Resources. Для полного комплекса 12 ГБ не являются проверенным минимумом.

СОЗДАНИЕ ФАЙЛА НАСТРОЕК

Терминал: папка проекта rag-test

ВЫПОЛНИТЬ В ТЕРМИНАЛЕ

```
git clone <адрес-репозитория> rag-test
cd rag-test
make env
nano infra/.env
```

Создайте окружение

make env копирует шаблон и заполняет пустые локальные пароли. Уже заданные непустые значения сохраняются.

Откройте файл

В nano найдите нужную строку и замените значение после =. Сохранить: Ctrl+O, Enter. Выйти: Ctrl+X.

Где взять подсказки

Полный перечень настроек и комментарии находятся в infra/.env.example. Рабочие значения задаются в infra/.env.

Команды показаны для новой установки. Если проект уже получен, начните с перехода в его папку.

ПАРОЛИ, КОТОРЫЕ СОЗДАЁТ MAKE ENV

Переменные	Для чего нужны
JWT_SECRET **ADMIN_PASSWORD**	Подпись сеансов и первый вход администратора. Имя задаёт ADMIN_USERNAME, по умолчанию admin.
POSTGRES_PASSWORD **RABBITMQ_PASSWORD**	Доступ к базе данных и очереди задач. Compose подставляет их во внутренние подключения.
S3_ACCESS_KEY **S3_SECRET_KEY**	Доступ к хранилищу загруженных файлов. Создаются локально, облачный аккаунт S3 не требуется.
GRAFANA_ADMIN_PASSWORD **RENDER_TOKEN**	Вход в Grafana и доступ к изолированному браузеру, который читает веб-источники.

Сохраните infra/.env в защищённом месте. Смена пароля в файле сама по себе не меняет пароль уже созданной базы.

ЛОКАЛЬНЫЙ ЗАПУСК ПО HTTP

Файл: infra/.env

ЗАПИСАТЬ В ФАЙЛ

```
FRONTEND_PORT=5173
HTTP_PORT=8080
AUTH_COOKIE_SECURE=false
AUTH_ALLOWED_ORIGINS=http://localhost:5173
WS_ALLOWED_ORIGINS=http://localhost:5173
FRONTEND_PUBLIC_API=
S3_PUBLIC_ENDPOINT=http://localhost:4566
REGISTRATION_ADMIN_ONLY=true
```

Когда подходит

Браузер открыт на той же машине, что и Docker. Вход: http://localhost:5173.

Если порт занят

Поменяйте FRONTEND_PORT и тот же порт в двух строках ORIGINS. Например, используйте 5174.

Доступ пользователей

При true учётные записи создаёт администратор. Публичное развёртывание настройте по разделу HTTPS.

AUTH_COOKIE_SECURE=false предназначено для локального HTTP. Для публичного сервера используйте HTTPS и true.

КАКОЙ АДРЕС УКАЗЫВАТЬ

Где находится сервис	Как записать адрес
В этом же Docker Compose	Используйте имя сервиса: http://llama-server:8090. localhost внутри контейнера указывает на сам контейнер.
На Mac с Docker Desktop	Для модели, запущенной на самом Mac: http://host.docker.internal:<порт>. Сервер модели должен принимать соединения из Docker.
На другой машине	Укажите доступное из Docker имя или IP и порт. Для внешнего провайдера используйте HTTPS.
В браузере пользователя	Адрес приложения и S3 должны открываться с компьютера пользователя. localhost чужого сервера для этого не подходит.

VLLM_URL задаётся без /v1. EMBEDDINGS_URL и RERANKER_URL, наоборот, включают полный путь метода.

КАКИЕ МОДЕЛИ НУЖНЫ КОМПЛЕКСУ

Задача	Что подключается
Тексты и карточки	LLM через VLLM_*. Нужна для открытого поиска радара и генерации ответов. Подключите свой сервер или облачного провайдера.
Поиск по смыслу	F2LLM создаёт числовое представление текста. Compose запускает f2llm-service, веса скачиваются при первом старте.
Документы и сканы	Реранкер уточняет порядок фрагментов. PaddleOCR-VL читает сканы. Нужны полному комплексу, а не режиму laptop.
Оценка слабого сигнала	Модель этапа 1 встроена в main-service. Отдельный API-ключ и отдельная установка этой модели не нужны.

Наличие GPU не подключает LLM автоматически. VLLM_URL нужно задать в любом варианте запуска.

ПОДКЛЮЧЕНИЕ ВНЕШНЕЙ LLM

Файл: infra/.env

ЗАПИСАТЬ В ФАЙЛ

```
VLLM_URL=https://<адрес-шлюза>
VLLM_MODEL=<точное-имя-модели>
VLLM_API_KEY='<ваш-API-ключ>'
LLM_MODEL_DISPLAY_NAME='<имя для пользователя>'
```

Получите доступ

У провайдера или владельца сервера запросите адрес OpenAI-совместимого API, имя модели и ключ.

Заполните четыре строки

URL без /v1 и без /chat/completions. MODEL содержит точный идентификатор из каталога. DISPLAY_NAME задаёт подпись в интерфейсе.

Уточните совместимость

Клиент использует chat/completions, max_tokens и temperature. Поддержка только Responses API не подходит.

Название переменных VLLM_* не требует установки vLLM. Это настройки любого совместимого сервера генерации.

ПРИМЕР: YANDEX AI STUDIO

Файл: infra/.env

ЗАПИСАТЬ В ФАЙЛ

```
VLLM_URL=https://llm.api.cloud.yandex.net
VLLM_MODEL=gpt://<folder_id>/yandexgpt/latest
VLLM_API_KEY='<ключ-сервисного-аккаунта>'
LLM_MODEL_DISPLAY_NAME='YandexGPT'
RADAR_LLM_WORKERS=8
```

Подготовьте аккаунт

Получите folder_id, API-ключ и право на вызов выбранной модели. Включите нужный доступ и оплату у провайдера.

Сверьте адрес и модель

Здесь приведён пример из репозитория. Скопируйте актуальные адрес и URI из своего кабинета. Если адрес заканчивается /v1, удалите этот суффикс.

Ограничьте параллельность

8 здесь только пример. Поставьте значение не выше разрешённого числа одновременных генераций. Затем выполните проверку.

Доступные модели и адрес API могут меняться. Решающий шаг перед запуском: bash scripts/check-llm.sh.

ПРОВЕРКА АДРЕСА, КЛЮЧА И МОДЕЛИ

Терминал: папка проекта rag-test

ВЫПОЛНИТЬ В ТЕРМИНАЛЕ

```
bash scripts/check-llm.sh
```

Что делает проверка

Читает VLLM_* из infra/.env и отправляет один короткий запрос. Нужен Python 3. Ключ и ответ не печатает.

Что означает успех

Должна появиться строка OK: non-empty completion received. Сообщение models endpoint unavailable допустимо, если эта строка появилась.

Если проверка не прошла

Проверьте адрес, модель, срок ключа и квоту. Внутренний llama-server проверяйте через Docker и запрос в интерфейсе.

Проверка расходует один вызов модели. Успех с хоста ещё не доказывает доступность адреса из контейнеров.

ЛОКАЛЬНАЯ LLM В DOCKER

Файл: infra/.env

ЗАПИСАТЬ В ФАЙЛ

```
LLAMA_GGUF_DIR=../models/gguf
LLAMA_GGUF_MODEL=Qwen3-30B-A3B-Q4_K_M.gguf
VLLM_URL=http://llama-server:8090
VLLM_MODEL=local-qwen
VLLM_API_KEY=
LLM_MODEL_DISPLAY_NAME='Qwen, свой сервер'
RADAR_LLM_WORKERS=1
```

Подготовьте GPU и веса

Нужна NVIDIA. Положите совместимый GGUF-файл в models/gguf. Автоматической загрузки этого файла нет.

Согласуйте имена

Имя файла задаёт LLAMA_GGUF_MODEL. Псевдоним API задаёт VLLM_MODEL. Путь каталога считается от infra.

Начните с одного вызова

Значение 1 ограничивает нагрузку при проверке. Повышайте его после настройки числа слотов сервера и оценки памяти.

Указанная GGUF-модель служит примером. Проверьте её совместимость и объём памяти с учётом остальных моделей.

ЗАПУСК ЛОКАЛЬНОЙ LLM

Терминал: папка проекта rag-test

ВЫПОЛНИТЬ В ТЕРМИНАЛЕ

```
COMPOSE_PROFILES=local-llm make up
make ps

# Только на работающем GPU-сервере:
COMPOSE_PROFILES=local-llm make deploy \
  SERVICES="llama-server"
```

Новая установка

Первая команда запускает полный комплекс вместе с llama-server. До неё подготовьте также веса OCR из раздела «Остальные модели».

Уже работающий сервер

Точечная команда запускает только модель. После изменения VLLM_* пересоздайте использующие их сервисы, как показано в разделе обслуживания.

Проверьте работу

В make ps найдите llama-server: состояние healthy. Затем выполните один запрос в радаре после готовности снимка источников.

На Mac модель можно запустить вне Docker через llama.cpp и использовать host.docker.internal. Docker-профиль требует NVIDIA.

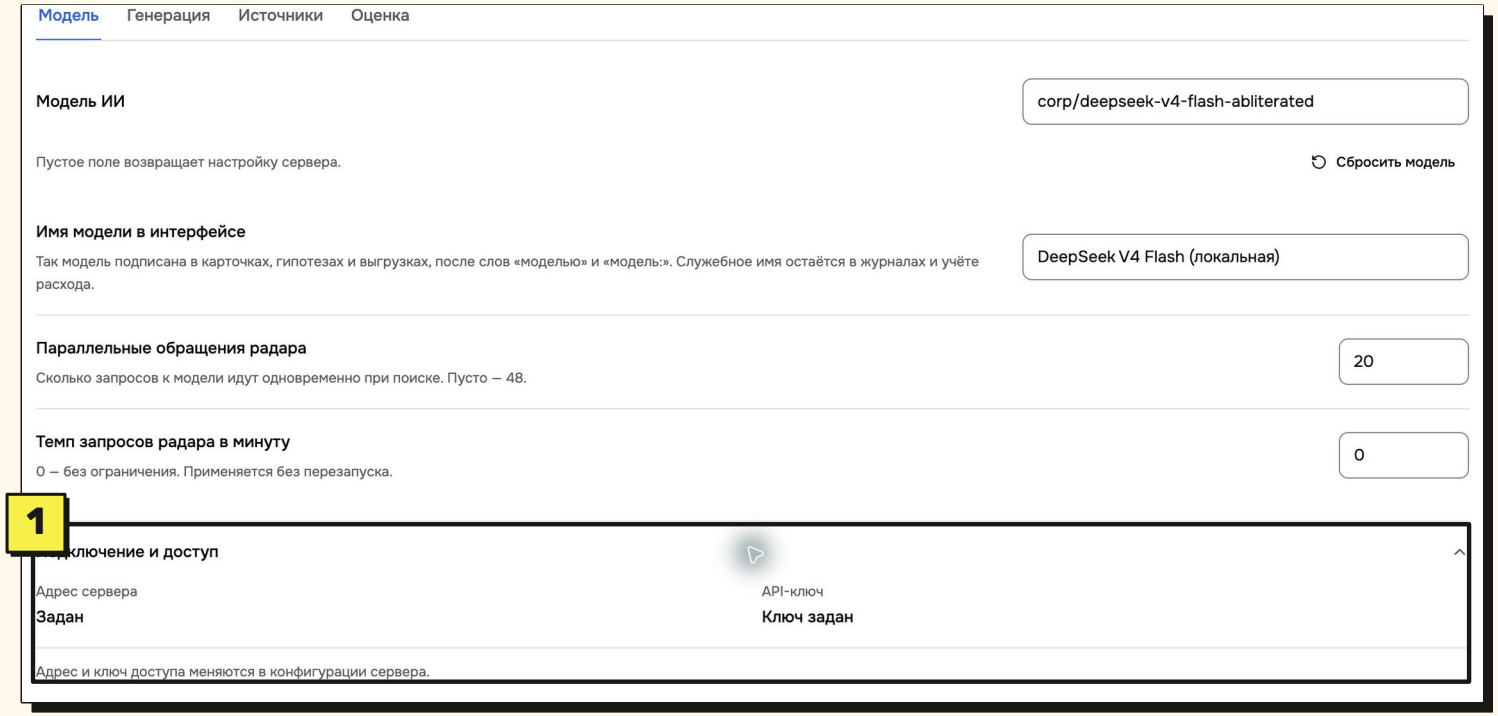
НАГРУЗКА И УЧЁТ РАСХОДОВ LLM

Настройка	Как выбрать значение
RADAR_LLM_WORKERS	Число одновременных обращений радара. Согласуйте с квотой провайдера или числом слотов локальной модели.
RADAR_LLM_RPM **VLLM_RPM**	Темп запросов в минуту у радара и клиента генерации. 0 снимает соответствующее ограничение. Начните с квоты вашего сервиса.
LLM_PRICES **LLM_COST_CURRENCY**	Прайс за 1 млн входных и выходных токенов и валюта. Пример формата: модель=цена_входа/цена_выхода. Возьмите цены у провайдера.
LLM_BUDGET_TOTAL **LLM_DAILY_LIMIT**	Бюджет и шкала запросов в «Метриках». Эти поля не гарантируют остановку платных запросов. Лимит расходов задайте у провайдера.

После смены LLM перепроверьте качество на /trends/model: оценка зрелости LLM входит в признаки модели этапа 1.

НАСТРОЙКИ МОДЕЛИ В ИНТЕРФЕЙСЕ

Управление ⟶ Настройки ⟶ Модель



1 **Адрес и ключ**

В «Подключение и доступ» виден только статус. Эти значения меняются в infra/.env на сервере.

2 **Имя и параметры**

«Модель ИИ», её подпись и параллельность можно изменить здесь. После правок нажмите появившуюся кнопку «Сохранить».

3 **Приоритет настроек**

Сохранённое имя модели может перекрывать значение из файла. «Сбросить модель» и затем «Сохранить» возвращают настройку сервера.

Настройки общие для всех пользователей. Снимок показывает пример работающего стенда, его имя модели копировать не нужно.

КАКИЕ КЛЮЧИ ПОДГОТОВИТЬ

Источник	Что даёт доступ
OpenAlex **OPENALEX_API_KEY**	Публикации и цитируемость. Ключ увеличивает доступный объём обращений. Рекомендуется для рабочего радара.
GitHub **GITHUB_TOKEN**	Поиск открытых проектов с более высоким лимитом. Без токена действует анонимный доступ и его ограничения.
EPO OPS и Роспатент	EPO_OPS_KEY вместе с EPO_OPS_SECRET; ROSPATENT_API_KEY. Без доступа соответствующий патентный канал отключён.
Product Hunt и Materials Project	PRODUCTHUNT_TOKEN для запусков продуктов. MP_API_KEY для проверок по материалам в фабрике гипотез. Подключайте при необходимости.

make env не выдаёт ключи внешних сервисов. Их нужно получить у поставщиков и вписать самостоятельно.

OPENALEX: ПУБЛИКАЦИИ И ЦИТИРУЕМОСТЬ

Файл: infra/.env

ЗАПИСАТЬ В ФАЙЛ

```
OPENALEX_API_KEY='<ваш-ключ>'
OPENALEX_MAILTO=admin@example.org
PUBSEARCH_MAILTO=admin@example.org
PUBSEARCH_ENABLED=true
```

Получите ключ

Создайте аккаунт OpenAlex. Откройте openalex.org/settings/api и скопируйте API key в OPENALEX_API_KEY.

Укажите контакт

Вместо admin@example.org укажите свою рабочую почту. MAILTO содержит контактный адрес. PUBSEARCH_ENABLED включает поиск мировой практики.

Проверьте источник

Примените изменения, выполните поиск и откройте оговорки сбора. Лимиты и расход проверьте в кабинете OpenAlex.

Без ключа доступ ограничен. Не фиксируйте квоту навсегда: её актуальное значение показывает сам OpenAlex.

GITHUB: ОТКРЫТЫЕ ПРОЕКТЫ

Файл: infra/.env

ЗАПИСАТЬ В ФАЙЛ

```
GITHUB_TOKEN='<персональный-токен>'
```

Создайте токен

GitHub: Settings → Developer settings → Personal access tokens. Выберите токен с доступом, достаточным для чтения публичных данных.

Сохраните в окружении

Вставьте токен в GITHUB_TOKEN. Для этой функции не нужны права записи в репозитории или администрирования организации.

Проверьте выдачу

Примените изменения. В новом поиске радара откройте источники GitHub и оговорки. При отказе проверьте срок и разрешения токена.

Здесь GITHUB_TOKEN является вашим токеном API. Автоматический токен GitHub Actions для развёртывания не требуется.

EPO OPS: МИРОВЫЕ ПАТЕНТЫ

Файл: infra/.env

ЗАПИСАТЬ В ФАЙЛ

```
EPO_OPS_KEY='<Consumer-Key>'
EPO_OPS_SECRET='<Consumer-Secret>'
```

Зарегистрируйте приложение

Откройте страницу Open Patent Services на epo.org. Перейдите в Register, войдите и создайте приложение.

Перенесите два значения

Consumer Key впишите в EPO_OPS_KEY. Consumer Secret впишите в EPO_OPS_SECRET. Нужны оба значения одного приложения.

Проверьте патенты

После применения настроек дождитесь сбора источника. В поиске проверьте патентные материалы и сообщения об ограничениях.

Без этой пары мировой канал EPO OPS не собирается. Его отсутствие не означает, что по теме нет патентов.

РОСПАТЕНТ: РОССИЙСКИЕ ПУБЛИКАЦИИ

Файл: infra/.env

ЗАПИСАТЬ В ФАЙЛ

```
ROSPATENT_API_KEY='<ваш-API-ключ>'
```

Оформите доступ

Откройте searchplatform.rospatent.gov.ru. Получите API-ключ в личном кабинете поисковой платформы.

Добавьте ключ

Вставьте выданное значение в ROSPATENT_API_KEY. Ключ другого патентного сервиса для этого поля не подходит.

Дождитесь сбора

Примените настройки и проверьте российские публикации в патентных источниках радара. При отказе сверьте доступ в кабинете поставщика.

Конфигурация сборщика использует набор ru_since_1994. Без ключа этот российский канал пропускается.

ДОПОЛНИТЕЛЬНЫЕ ИСТОЧНИКИ

Переменная	Где взять и зачем
PRODUCTHUNT_TOKEN	Developer token в API dashboard Product Hunt. Даёт сведения о запусках продуктов. Для коммерческого использования согласуйте доступ с Product Hunt.
MP_API_KEY	Ключ в профиле Materials Project. Нужен для проверок новизны материалов в фабрике гипотез. Без ключа реальный MP-поиск не выполняется.
SEC_EDGAR_USER_AGENT	Впишите организацию и контактную почту, например: «Компания admin@example.org». Это идентификация клиента, не секретный ключ.
Источники без ключей	Для arXiv, Hacker News и части открытых реестров отдельные ключи в этой конфигурации не предусмотрены. Им всё равно нужен сетевой доступ.

Пустой ключ отключает или ограничивает только соответствующий источник. Учитывайте это при чтении результатов.

ПРОКСИ И КОРПОРАТИВНЫЙ СЕРТИФИКАТ

Файл: infra/.env

ЗАПИСАТЬ В ФАЙЛ

```
SOURCE_PROXY_URL=
OPENALEX_PROXY_URL=
RAG_CA_BUNDLE=
```

Обычное подключение

Оставьте строки пустыми, если источники и LLM доступны напрямую и сертификаты доверенные.

Прокси источников

SOURCE_PROXY_URL задаёт прокси сборщиков. OPENALEX_PROXY_URL задаёт отдельный прокси OpenAlex. Формат: http://host:port или socks5://host:port.

Свой центр сертификации

Для корпоративной LLM впишите в RAG_CA_BUNDLE абсолютный путь к PEM с корпоративными и публичными корнями. Make подключит нужный оверлей.

Эти прокси не являются общей настройкой сети Docker. Для check-llm.sh корпоративный CA задайте также через SSL_CERT_FILE.

F2LLM: ПОИСК ПО СМЫСЛУ

Файл: infra/.env

ЗАПИСАТЬ В ФАЙЛ

```
F2LLM_MODEL=codefuse-ai/F2LLM-v2-0.6B
EMBEDDINGS_MODEL=codefuse-ai/F2LLM-v2-0.6B
EMBEDDING_DIM=1024
EMBEDDINGS_URL=http://f2llm-service:8085/v1/embeddings
```

Оставьте стандартную модель

Эти значения уже есть в шаблоне. Они нужны и радару, и поиску по документам. API-ключ не требуется.

Дождитесь загрузки

При первом запуске f2llm-service скачивает веса в models/huggingface. В make ps дождитесь healthy.

CPU или GPU

make laptop и CPU-вариант make bootstrap переводят сервис на CPU. Для GPU используется F2LLM_TORCH_DEVICE=cuda.

EMBEDDINGS_MODEL описывает модель для клиента. F2LLM_MODEL выбирает реальные веса сервера. Значения должны совпадать.

ЗАГРУЗКА ВЕСОВ И ДОСТУП HUGGING FACE

1	**Открытые модели**	Стандартные F2LLM и Qwen Reranker скачиваются при запуске. Их кэш находится в models/huggingface. Файлы OCR загружаются отдельным шагом.
2	**Закрытый репозиторий**	Если вы выбрали модель с ограниченным доступом, сначала получите разрешение владельца. Загрузчик hf можно авторизовать командой hf auth login.
3	**Передача доступа в Docker**	Один HF_TOKEN в infra/.env не решает задачу: стандартный Compose его не передаёт. Для закрытой модели настройте передачу секрета и кэш с разработчиком.

На чистом сервере проверьте сеть, место на диске и доступ к каталогу models. Ошибку загрузки весов ищите в логах модели.

ЗАМЕНА МОДЕЛИ ЭМБЕДДИНГОВ

1	**Когда это отдельная работа**	Для первого запуска сохраните F2LLM из шаблона. Замена модели меняет пространство векторов, поэтому старый индекс с новой моделью несовместим.
2	**Что согласовать**	Нужны единая модель у всех клиентов, EMBEDDING_DIM, полная переиндексация и переобучение модели этапа 1. Некоторые адреса воркеров закреплены в Compose.
3	**Внешний сервис**	Один EMBEDDINGS_URL не переключает весь комплекс. Согласуйте изменения Compose с разработчиком. Для закреплённой модели сверьте F2LLM_REVISION и EMBEDDINGS_REVISION.

Если меняли модель или её режим, сверьте revision из /health сервиса и проверьте контрольный поиск до подключения пользователей.

РЕРАНКЕР: ТОЧНОСТЬ ПОИСКА ДОКУМЕНТОВ

Файл: infra/.env

ЗАПИСАТЬ В ФАЙЛ

```
RERANKER_SERVICE_MODEL=Qwen/Qwen3-Reranker-0.6B
RERANKER_MODEL=Qwen/Qwen3-Reranker-0.6B
RERANKER_URL=http://reranker-service:8086/v1/rerank
```

Для чего нужен

Уточняет порядок найденных фрагментов документов. В полном комплексе Compose запускает reranker-service автоматически.

Стандартный запуск

Оставьте значения шаблона. Дождитесь загрузки весов и healthy. Для laptop сервис выключен.

Если подключаете свой

Нужен совместимый POST /v1/rerank с query, documents и оценками. После замены проверьте пороги качества на своих документах.

Отдельного RERANKER_API_KEY в этой конфигурации нет. Сервис с другим протоколом или авторизацией потребует адаптации.

OCR: ФАЙЛЫ МОДЕЛИ ДО ПЕРВОГО ЗАПУСКА

Терминал: папка проекта rag-test

ВЫПОЛНИТЬ В ТЕРМИНАЛЕ

```
python3 -m venv .venv-hf
.venv-hf/bin/pip install huggingface_hub
.venv-hf/bin/hf download \
  PaddlePaddle/PaddleOCR-VL-1.6-GGUF \
  --local-dir models/gguf/paddlevl
```

Для полного комплекса

OCR читает сканы и изображения. В текущем Compose распознавание выполняет paddleocr-llama. Ему нужны GGUF-файлы на диске.

Скачайте до старта

Команды создают отдельную среду для загрузчика и скачивают модель. Для режима laptop этот шаг можно пропустить.

Проверьте файлы

В каталоге должны быть PaddleOCR-VL-1.6-GGUF.gguf и PaddleOCR-VL-1.6-GGUF-mmproj.gguf. Имена должны совпадать с Compose.

Автоматическая загрузка остальных весов не заменяет этот шаг. Отсутствие GGUF-файлов остановит готовность OCR.

OCR: ПОДКЛЮЧЕНИЕ И ПРОВЕРКА

Файл: infra/.env

ЗАПИСАТЬ В ФАЙЛ

```
IMAGE_PARSE_ROUTE=ocr
OCR_ENGINE_URL=http://paddleocr-vl-service:8088
OCR_MODEL=PaddleOCR-VL

# Только при других именах файлов:
PADDLEOCR_VL_GGUF=paddlevl/<файл>.gguf
PADDLEOCR_VL_GGUF_MMPROJ=paddlevl/<mmproj>.gguf
```

Оставьте стандартный маршрут

Значение ocr направляет загруженные изображения в локальное распознавание. Разметка страницы скачает свои веса в models/paddlex.

Проверьте два контейнера

paddleocr-llama и paddleocr-vl-service должны стать healthy. На CPU распознавание может занимать больше времени.

Проверьте документ

Загрузите небольшой скан через «Документы». Дождитесь обработки и проверьте, что распознанный текст доступен в поиске.

При изменении LLAMA_GGUF_DIR сохраняйте внутри него каталог paddlevl: генерация и OCR используют общую папку GGUF.

VLM: ОПИСАНИЕ СХЕМ И ИЗОБРАЖЕНИЙ

Файл: infra/.env

ЗАПИСАТЬ В ФАЙЛ

```
IMAGE_PARSE_ROUTE=vlm
VLM_ENGINE_URL=https://<адрес-шлюза>
VLM_API_STYLE=openai
VLM_MODEL=<модель-с-поддержкой-изображений>
VLM_API_KEY='<ключ-для-этой-модели>'
```

Когда нужно

Подключайте VLM, если требуется смысловое описание изображений через мультимодальную модель. Обычный запуск использует OCR.

Заполните отдельные поля

Текстовые VLLM_* не заменяют VLM_*. Укажите модель с поддержкой image_url. Для стиля openai URL задаётся без /v1.

Проверьте результат

Примените настройки и загрузите тестовую схему. Оцените, верно ли описаны подписи и связи. Внешний сервис получает изображение.

VLM_API_STYLE=native использует другой протокол: model, image_b64, mime, prompt → text. Выбирайте стиль своего сервера.

НЕОБЯЗАТЕЛЬНЫЙ EXTRACTOR

Файл: infra/.env

ЗАПИСАТЬ В ФАЙЛ

```
RADAR_CANDIDATE_SOURCE=ml
RADAR_ML_EXTRACT_URL=http://extractor:8080/extract
RADAR_CLASSIFIER_URL=
RADAR_CLASSIFIER_VERSION=<service_version-бандла>
EXTRACTOR_MODEL_BUNDLE_URL=<адрес-архива>
EXTRACTOR_MODEL_BUNDLE_SHA256=<SHA-256-архива>
EXTRACTOR_EMBEDDING_URL=http://f2llm-service:8085
```

Что меняется

Extractor выделяет технологии из текста. Он не заменяет LLM и встроенную модель этапа 1. Для первого запуска можно оставить ngrams.

Что получить

У владельца моделей запросите архив, его SHA-256 и service_version из release.json. Эти три значения должны относиться к одному выпуску.

Где запускается

Профиль ml-extractor требует NVIDIA в стандартной конфигурации. Бандл скачивается в том extractor-models.

RADAR_CLASSIFIER_URL здесь оставляют пустым. Вердикт даёт встроенная модель main-service.

ЗАПУСК EXTRACTOR И ДРУГИЕ ПРОФИЛИ

Терминал: папка проекта rag-test

ВЫПОЛНИТЬ В ТЕРМИНАЛЕ

```
COMPOSE_PROFILES=ml-extractor make deploy \
  SERVICES="extractor"
make deploy SERVICES="main-service"
make ps
```

Extractor на GPU-сервере

Нужен работающий полный комплекс с PostgreSQL и F2LLM. После команд дождитесь healthy у extractor.

Специальные варианты OCR

ocr-vllm и diagram-parser-paddle5 требуют GPU. diagram-parser разбирает схемы локально. Базовому запуску они не нужны.

Загрузка научного корпуса

Для signalgraph задайте HARVEST_OWNER_ID: UUID существующего оператора. Профиль загружает научные документы в базу.

При первом запуске extractor может долго загружать веса. Стандартный healthcheck даёт ему до 15 минут на прогрев.

ПЕРВЫЙ ЗАПУСК

Выполните один вариант	Когда использовать
make laptop	Только радар на CPU. Документы, распознавание и метрики в этом варианте не запускаются.
make bootstrap	Полный комплекс. Автоматически выбирает CPU, если NVIDIA runtime недоступен. До запуска скачайте файлы OCR.
make up	Полный комплекс с NVIDIA. До запуска подготовьте Container Toolkit и файлы OCR.
COMPOSE_PROFILES=local-llm make up	Полный комплекс и локальная LLM. Нужны NVIDIA, GGUF генерации и файлы OCR.

Перед запуском: make config → compose OK. После запуска: make ps. Проверка конфигурации не проверяет ключи API.

ГОТОВНОСТЬ ИНТЕРФЕЙСА И ДАННЫХ

Терминал: папка проекта rag-test

ВЫПОЛНИТЬ В ТЕРМИНАЛЕ

```
curl http://localhost:5173/healthz
make ps
make logs
```

Интерфейс

Ответ ok подтверждает доступность веб-сервера. Он ещё не означает готовность всех моделей и источников. При другом порте измените адрес.

Первый снимок

Сообщение 503 live snapshot is still being collected означает, что радар собирает начальные данные. Смотрите логи radar-collector.

Время ожидания

Первый сбор может занять больше часа. В проверке репозитория окно arXiv собиралось 86 минут. Это пример замера, а не обещанный срок.

make wait-healthy пропускает сборщики. Их временное unhealthy во время первого сбора не равно готовности радара.

ПЕРВЫЙ ВХОД И ПРОВЕРКА ФУНКЦИЙ

1	**Вход**	Откройте http://localhost:5173. Введите ADMIN_USERNAME и ADMIN_PASSWORD из infra/.env. При изменённом порте используйте свой адрес.
2	**Радар**	После готовности источников задайте знакомое направление. Дождитесь выдачи, откройте карточку и оговорки сбора. Проверьте, какая модель указана в результате.
3	**Полный комплекс**	Загрузите небольшой текстовый PDF и скан. Проверьте обработку в «Документах», поиск по ним и открытие источника ответа.

Успешный запуск контейнеров не гарантирует качество карточек. Перед рабочим использованием проверьте результаты на знакомых примерах.

ПУБЛИЧНЫЙ СЕРВЕР: HTTPS

Файл: infra/.env

ЗАПИСАТЬ В ФАЙЛ

```
FRONTEND_TLS_CERT=/private/fullchain.pem
FRONTEND_TLS_KEY=/private/private.key
FRONTEND_TLS_BIND=0.0.0.0
FRONTEND_HTTPS_PORT=8443
S3_HTTPS_PORT=8444
STAND_URL=https://radar.example.org:8443
S3_PUBLIC_ENDPOINT=https://radar.example.org:8444
AUTH_COOKIE_SECURE=true
AUTH_ALLOWED_ORIGINS=https://radar.example.org:8443
WS_ALLOWED_ORIGINS=https://radar.example.org:8443
```

Подготовьте домен

Направьте DNS на сервер. Получите сертификат с полной доверенной цепочкой. Храните сертификат и ключ вне репозитория.

Замените пример

Укажите свои пути и домен. FRONTEND_PUBLIC_API оставьте пустым. Make автоматически подключит HTTPS-оверлей.

Проверьте с другого ПК

Вход, обновление сеанса, загрузка файла и его открытие должны работать по HTTPS без предупреждений браузера.

0.0.0.0 открывает HTTPS на всех сетевых интерфейсах. Ограничьте доступ правилами сети, а прямой HTTP не публикуйте наружу.

КАК ПРИМЕНИТЬ НОВЫЕ НАСТРОЙКИ

1	**Сначала сохраните файл**	Измените infra/.env. Проверьте его командой make config. Для новой внешней LLM дополнительно выполните bash scripts/check-llm.sh.
2	**Пересоздайте сервисы**	На своей новой установке повторите выбранную команду запуска. Обычный docker restart не перечитывает переменные окружения.
3	**На рабочем сервере**	Выполните точечную пересборку только затронутых сервисов. Список зависит от изменения. Учитывайте CPU-оверлей, профили, HTTPS и корпоративный CA.

В интерфейсе сохранённая модель может перекрывать VLLM_MODEL из файла. Сверьте раздел «Настройки → Модель».

КАКИЕ СЕРВИСЫ ИСПОЛЬЗУЮТ НАСТРОЙКИ

Что изменили	Какие сервисы проверить и пересоздать
LLM: VLLM_*	main-service, llm-service, radar-collector, chunk-splitter, cluster-worker, discovery-worker, raptor-worker, eval-service. Только включённые в вашем варианте.
OpenAlex и GitHub	main-service, radar-collector, radar-analysis. При использовании дополнительных профилей проверьте также их окружение.
EPO OPS и Роспатент	patent-collector. Затем дождитесь нового сбора и проверьте сообщения источника.
VLM и HTTPS	VLM_*: vlm-service. IMAGE_PARSE_ROUTE: db-service, main-service и source-harvester, если он включён. HTTPS: следуйте docs/HTTPS.md.

make deploy подходит для GPU-контура. На CPU сохраните оверлей docker-compose.nogpu.yml при прямом запуске Compose.

ПРИМЕР: ОБНОВЛЕНИЕ LLM В РЕЖИМЕ LAPTOP

Терминал: папка проекта rag-test

ВЫПОЛНИТЬ В ТЕРМИНАЛЕ

```
docker compose --env-file infra/.env \
  -f infra/docker-compose.yml \
  -f infra/docker-compose.nogpu.yml \
  -f infra/docker-compose.laptop.yml \
  up -d --build --no-deps \
  main-service llm-service radar-collector
```

Когда подходит

Вы изменили VLLM_* на работающей локальной установке laptop. Команда обновляет три сервиса с сохранением CPU-настроек.

Сохраните свои оверлеи

Если используете HTTPS, корпоративный CA или docker-compose.local.yml, добавьте их -f в том же порядке, что при запуске.

Проверьте после обновления

Выполните make ps. Затем запустите запрос в интерфейсе и проверьте имя модели. Убедитесь, что настройки интерфейса не перекрывают файл.

Для полного CPU-комплекса уберите laptop-оверлей и добавьте включённые сервисы LLM из таблицы на предыдущей странице.

ЕСЛИ ВХОД ИЛИ LLM НЕ РАБОТАЮТ

Что вы видите	Что проверить
Не открывается приложение	Docker запущен, frontend работает, порт FRONTEND_PORT свободен. Проверьте адрес и curl /healthz.
Сеанс пропадает на HTTP	В локальном окружении задайте AUTH_COOKIE_SECURE=false. Для внешнего доступа настройте HTTPS и верните true.
LLM: 401 или 403	Проверьте ключ, срок действия, права на выбранную модель и доступ аккаунта. Повторите check-llm.sh после исправления.
LLM: 404, таймаут или пустой ответ	Проверьте базовый URL без /v1, имя модели и доступ из Docker. Уточните совместимость max_tokens и temperature, квоту и доверие сертификату.

Ключи и полный infra/.env не отправляйте в переписку. Для помощи достаточно времени ошибки, имени сервиса и текста без секретов.

ЕСЛИ МОДЕЛИ И ИСТОЧНИКИ НЕ ГОТОВЫ

Что вы видите	Следующий шаг
Контейнеры перезапускаются	Проверьте память Docker и свободный диск. Для laptop нужно не менее 12 ГБ памяти Docker. Ищите OOM в событиях и логах.
OCR не становится healthy	Проверьте оба GGUF-файла и LLAMA_GGUF_DIR. Затем проверьте paddleocr-llama и paddleocr-vl-service.
Extractor: версия не совпадает	Сверьте RADAR_CLASSIFIER_VERSION с service_version в release.json и SHA-256 архива. Не смешивайте настройки разных бандлов.
Источник недоступен или лимит исчерпан	Проверьте ключ, кабинет поставщика, сеть и прокси. Дождитесь новой квоты или сбора. Не трактуйте отсутствие данных как отсутствие технологии.

Логи всех контейнеров: make logs. Ctrl+C прекращает просмотр логов и не останавливает комплекс.

РЕЗЕРВНАЯ КОПИЯ И ОСТАНОВКА

Терминал: папка проекта rag-test

ВЫПОЛНИТЬ В ТЕРМИНАЛЕ

```
make bundle-export OUT=/private/radar-backup \
  FLAGS="--with-cache --with-models"
make bundle-verify BUNDLE=/private/radar-backup

# Обычная остановка:
make down
```

Выберите время

Экспорт останавливает комплекс. Предупредите пользователей. После копирования повторите свою команду запуска.

Что попадёт в копию

Данные, связанные секреты, кэши и каталог models. Копия содержит секреты: ограничьте доступ к ней.

Что сохранить отдельно

Полный infra/.env, сертификаты, оверлеи и версию кода. При extractor сохраните также его именованный том.

make down сохраняет данные. make clean удаляет тома. Скрипт бандла рассчитан на стандартное имя проекта rag-platform.

ПЕРЕНОС НА ДРУГУЮ МАШИНУ

Терминал: папка проекта rag-test

ВЫПОЛНИТЬ В ТЕРМИНАЛЕ

```
make env
make bundle-verify BUNDLE=/private/radar-backup
make bundle-import BUNDLE=/private/radar-backup
```

Подготовьте чистую установку

Используйте ту же версию кода и Compose. Скопируйте бандл на чистую машину. Импорт меняет данные.

Восстановите доступы

Импорт переносит связанные ключи. Внешние API-ключи, адреса, сертификаты и оверлеи восстановите отдельно.

Проверьте после старта

Повторите вход, запрос радара и открытие старого документа. Убедитесь, что LLM, источники и модель эмбеддингов соответствуют перенесённым данным.

Для extractor скрипт бандла не копирует том extractor-models. Нужна отдельная копия либо повторная загрузка согласованного бандла.

ЧТО ПЕРЕДАТЬ АДМИНИСТРАТОРУ

1	**Адрес и способ запуска**	URL приложения, выбранный режим, версия кода, список оверлеев и профилей. Отдельно укажите ограничения варианта laptop.
2	**Владельцы доступов**	Кто управляет доменом, сертификатом, LLM и ключами источников. Где защищённо хранятся секреты и когда истекают ключи.
3	**Проверки и восстановление**	Дата последней успешной проверки, пример контрольного запроса, место резервной копии и порядок возврата в работу.

Готовность к работе подтверждают вход, выдача радара и проверки нужных функций. Одного статуса healthy недостаточно.

ГДЕ ИСКАТЬ НУЖНУЮ ПЕРЕМЕННУЮ

Задача	Имена для поиска в PDF и infra/.env.example
Генерация и изображения	VLLM_URL, VLLM_MODEL, VLLM_API_KEY, LLM_MODEL_DISPLAY_NAME; VLM_ENGINE_URL, VLM_MODEL, VLM_API_STYLE, VLM_API_KEY.
Источники	OPENALEX_API_KEY, OPENALEX_MAILTO, PUBSEARCH_MAILTO, GITHUB_TOKEN, EPO_OPS_KEY, EPO_OPS_SECRET, ROSPATENT_API_KEY, PRODUCTHUNT_TOKEN, MP_API_KEY.
Локальные модели	F2LLM_MODEL, EMBEDDINGS_MODEL, EMBEDDING_DIM, RERANKER_SERVICE_MODEL, OCR_ENGINE_URL, LLAMA_GGUF_DIR, RADAR_ML_EXTRACT_URL.
Сеть и пользователи	FRONTEND_PORT, AUTH_COOKIE_SECURE, AUTH_ALLOWED_ORIGINS, WS_ALLOWED_ORIGINS, S3_PUBLIC_ENDPOINT, RAG_CA_BUNDLE, REGISTRATION_ADMIN_ONLY.

Для первого запуска остальные параметры оставьте из шаблона. Точный полный перечень находится в infra/.env.example.

ДОКУМЕНТЫ В РЕПОЗИТОРИИ

Файл	Когда открыть
README.md	Варианты запуска, ограничения текущей версии и назначение моделей.
infra/.env.example **infra/docker-compose.yml**	Все переменные, реальные зависимости и адреса сервисов. Дополнительные файлы docker-compose.*.yml меняют режим запуска.
docs/HTTPS.md **scripts/check-llm.sh**	Настройка публичного HTTPS и проверка внешней модели генерации.
ml-services/*/README.md **infra/scripts/bundle.sh**	Контракты и веса моделей. Состав резервной копии и правила переноса данных.

Редакция руководства: 27.09.2026, commit 4674a329. При обновлении кода повторно сверьте примеры с конфигурацией.

КАБИНЕТЫ И ДОКУМЕНТАЦИЯ СЕРВИСОВ

Нажмите на название. Полученные секреты вносите только в приватный файл настроек.

OpenAlex: API-ключ

openalex.org

OpenAlex: правила доступа

help.openalex.org

GitHub: токены API

docs.github.com

EPO: Open Patent Services

www.epo.org

Роспатент: поисковая платформа

searchplatform.rospatent.gov.ru

Product Hunt: API и условия

api.producthunt.com

Materials Project: API-ключ

docs.materialsproject.org

Hugging Face: загрузка весов

huggingface.co

Квоты, цены и перечень моделей проверяйте в кабинете выбранного поставщика перед подключением.